

Assignment Report on Advanced Multi-Client Chat Server using TCP

Submitted By	
Student Name:	Shahan Uz Zaman
Roll Number:	CS-BTC24-23
Course / Subject Name	Summer internship on Cyber Security and Secure App Development

1. Objective

The objective of this assignment is to enhance the multi-client TCP chat application developed in Assignment 4 by implementing advanced communication features including client management, private messaging, persistent chat history, performance evaluation, and automatic graph generation. The application is designed using Python socket programming and multithreading and is executed inside the Mininet virtual networking environment.

The objectives are:

Develop an advanced multi-client TCP chat server.

Manage multiple clients simultaneously using multithreading.

Maintain user information and connection status.

Implement private messaging between clients.

Display dynamically updated online user lists.

Store chat history in CSV format.

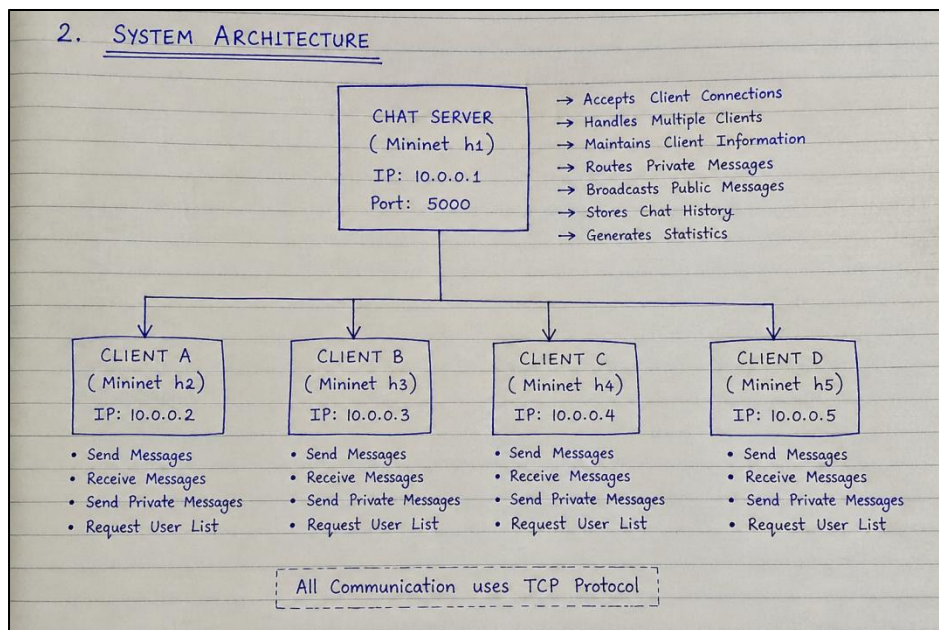
Evaluate communication performance.

Generate automatic performance graphs.

Verify TCP communication using Wireshark.

2. System Architecture

The system follows a client-server architecture.



Components

Server

- Listens on TCP Port 5000
- Accepts multiple clients
- Maintains client information
- Routes private messages
- Broadcasts public messages
- Stores chat history
- Generates statistics

Clients

- Connect to server
- Send broadcast messages
- Send private messages

- Request online user list
- Receive messages simultaneously

3. Design Improvements over Assignment 4

Compared to Assignment 4, the following improvements were implemented.

Assignment 4	Assignment 5
Basic multi-client chat	Enhanced client management
Broadcast messaging	Private messaging
No user database	Client information storage
No history	Persistent CSV history
No statistics	Performance measurement
No online list	Dynamic online user list

New Features

- Username-based login
- Join and leave notifications
- Online/offline status
- Private messaging (/msg)
- Online user list (/list)
- Chat history CSV
- Performance evaluation
-

4. Program Design

The implementation consists of three major modules.

Server Module

Responsibilities

- Accept TCP connections
- Handle multiple clients
- Manage usernames
- Route messages
- Broadcast notifications
- Store chat history
- Generate statistics

Main functions

- initialize_files()
- handle_client()
- broadcast()
- send_private()
- show_last_messages()

Client Module

Responsibilities

- Connect to server
- Register username

- Send messages
- Receive messages
- Display online users
- Send private messages

Commands

- /list
- /msg username message
- /exit

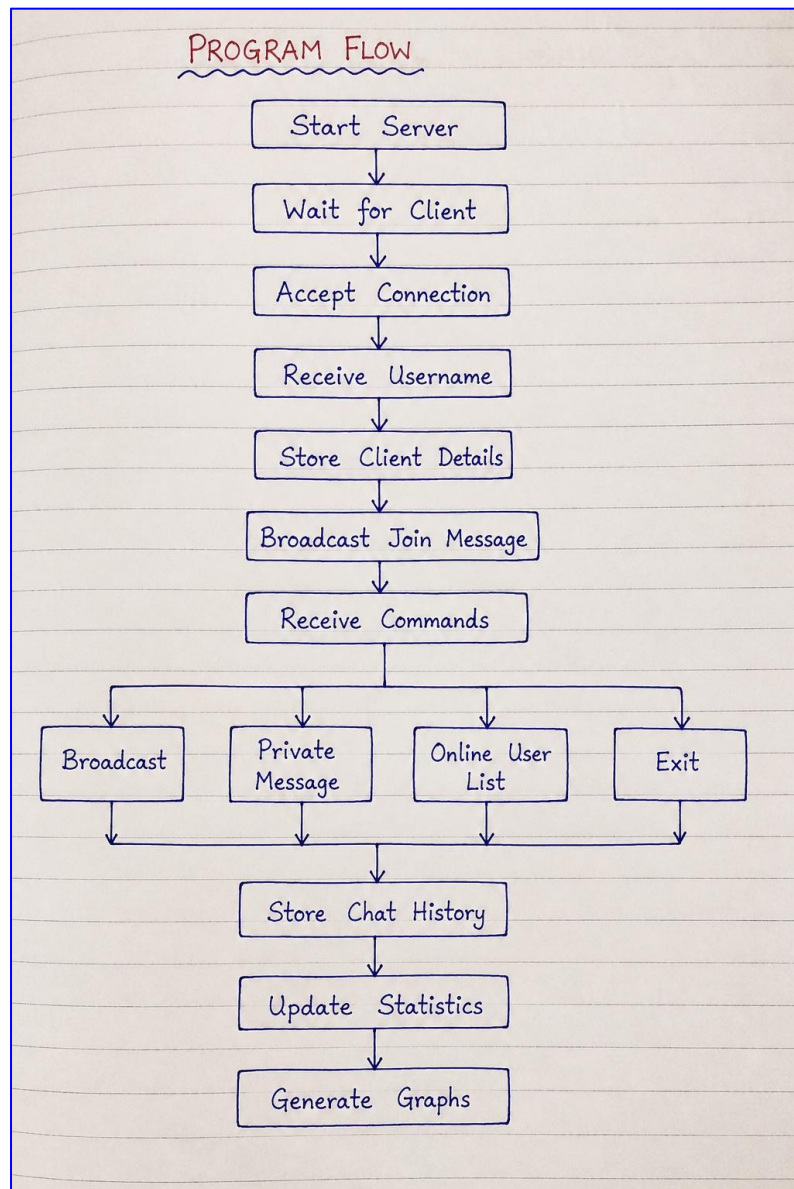
Graph Module

Reads

- performance_results.csv

Graph generates

- clients_vs_delay.png
- clients_vs_throughput.png
- message_type_distribution.png



5. Experimental Setup

Operating System

- Ubuntu Linux

Programming Language

- Python 3

Libraries

- socket
- threading
- csv
- datetime
- matplotlib
- pandas

Network Emulator

- Mininet

Network Topology

sudo mn --topo single,5

Topology

- h1 Server
- h2 Client A
- h3 Client B
- h4 Client C
- h5 Client D

Verification

- nodes
- net
- pingall

6. Results

performance_results.csv

```
clients,broadcast_messages,private_messages,avg_delay_ms,throughput_msgs_per_sec
1,54,1,2315.83,0.43
4,57,23,4590.69,0.22
```

7. Graph Analysis

Three graphs were generated automatically.

Clients vs Average Delivery Time

As the number of connected clients increased, the average message delivery time also increased due to additional server processing and communication overhead.

Clients vs Throughput

Throughput increased as more clients transmitted messages simultaneously. However, after a certain point, server processing overhead may reduce efficiency.

Broadcast Messages vs Private Messages

Broadcast messages represented the majority of network traffic because public communication occurred more frequently than private communication.

8. Wireshark Verification

TCP traffic was captured using the following display filter:

tcp.port == 5000

The following events were verified:

- Client connection establishment
- Broadcast message transmission
- Private message transmission
- Client disconnection
- TCP connection termination (FIN/ACK)

Screenshots included:

- Client Connection
- Broadcast Message
- Private Message
- Client Disconnect
- TCP FIN/ACK

9. Reflection Answers

1. Why is server-side routing necessary for private messaging?

Server-side routing ensures that private messages are delivered only to the intended recipient. The server identifies the destination user and forwards the message securely without broadcasting it to other connected clients.

2. How does maintaining user state improve the application?

Maintaining user state enables the server to track usernames, login times, IP addresses, online status, and message history, improving reliability and user management.

3. How would the application scale to 100 users?

The application could be scaled by using thread pools or asynchronous I/O, efficient message queues, optimized data structures, and deploying the server on more capable hardware.

4. Which Wireshark packets verified private messaging?

TCP packets carrying the private message payload between the sender and the server, followed by packets from the server to the intended recipient, confirmed correct private message routing.

5. What improvements would you implement next?

- End-to-end encryption
- File sharing
- Group chat
- Message acknowledgements
- Database storage
- Web-based user interface

10. Conclusion

This assignment successfully extended the Assignment 4 TCP chat application into a feature-rich multi-client communication system. The server efficiently managed multiple clients using multithreading, supported both broadcast and private messaging, maintained persistent chat history, and tracked client information. Performance evaluation was conducted for different numbers of clients, and the collected data was automatically converted into graphical reports. Wireshark verification confirmed correct TCP communication and private message routing. The completed system demonstrates practical implementation of client-server communication, concurrent programming, network

performance analysis, and TCP protocol behavior in a Mininet environment, meeting the objectives of the assignment.

